Pas2JS SPA Framework

Technical Documentation & Developer Guide

Version 1.1 — 2026

Table of Contents

1. Introduction
2. Overall Architecture
3. Framework Modules
3.1 WebComponent — UI Component Base
3.2 LayoutComponents — Page Structure
3.3 PageManager — SPA Routing
3.4 WebApplication — Application Shell
3.5 EventsManager — Internal Pub/Sub
3.6 InfiniteScroller — Progressive Loading
3.7 Dialogs — Modal Dialogs
4. CSS & Theming
5. Full Example: the “Address Book” App
6. Quick API Reference
7. **Other modules**

1. Introduction

This framework is a lightweight library for Free Pascal compiled with Pas2JS. It allows you to build Single-Page Applications (SPA) entirely in Pascal, without relying on any external JavaScript framework.

The primary goal is to provide an abstraction layer that respects the object-oriented paradigm typical of Delphi/FPC, while adapting it to the asynchronous, event-driven model of the modern browser.

Key Features

- **Hash-based SPA routing** with full browser Back button support
- Built-in responsive support: different form classes for desktop and mobile
- **Complete component lifecycle** management (Create, BindUI, Destroy)
- Asynchronous on-demand loading of HTML templates and CSS files
- Pub/sub event system for decoupled cross-component communication
- InfiniteScroller with template engine for large data lists
- Wake Lock API support for kiosk and touch-screen applications

- CSS variable-based theming with three built-in themes

> *Requirement: Free Pascal with Pas2JS 3.x or later. No npm or external JavaScript dependencies required.*

2. Overall Architecture

The framework follows a layered architecture with well-separated responsibilities. Each Pascal unit maps to a distinct logical layer of the application.

Layer	Unit	Responsibility
Shell	WebApplication	Entry point, layout coordination, startup sequence
Layout	LayoutComponents	Header, footer, and responsive side columns
Routing	PageManager	Hash-based navigation, screen-size form selection
Components	WebComponent	Base class for any UI component
Communication	EventsManager	Global pub/sub for cross-component events
Utility	InfiniteScroller	Infinite scroll with template engine and chunking
Utility	Dialogs	Modal message dialogs (single OK button)

The startup flow always follows the same sequence:

```
  rtl.run()                          // index.html calls the entry point
    └─ TMyApp.Create(...)               // creates the shell with layout classes
         └─ Run()                        // [async]
              ├─ InitializeLayout()     // loads header / footer / columns
              ├─ RegisterRoutes()       // maps routes to TBaseForm classes
              └─ StartRouting()         // reads the URL hash and loads the initial
page
```

3. Framework Modules

3.1 WebComponent — UI Component Base

The file **WebComponent.pas** defines the two fundamental classes on which the rest of the framework is built.

TWebComponent

Abstract base class representing any piece of UI associated with a root DOM element. It maintains an internal cache of child element wrappers, avoiding repeated DOM queries.

Member	Type	Description
Create(aRootElement)	constructor	Receives the root DOM element. Does not call BindUI: see the BindUI row
BindUI	procedure virtual	Override to bind events and initialize the UI. Called by PageManager after Create (pages) and by LoadContent after the markup is loaded. A component whose markup is already in the page must call it explicitly right after Create
getElementById(id)	function	Returns a TUIElementWrapper with lazy-init and caching
RootElement	property TJSHTMLElement	Read-only access to the root element

TUIElementWrapper

A wrapper around a single HTML element. All DOM interactions go through this class, which handles lazy element lookup and automatic event listener cleanup in its destructor.

Property / Method	Description
value	Reads/writes element value (input, select, textarea — unified)
innerHTML	Reads/writes the inner HTML of the element
checked	Reads/writes the checked state of a checkbox
disabled	Enables or disables the control
textContent	Writes plain text (no HTML markup)
onclick	Assigns a typed TUIElementClickEventHandler
onchange	Assigns a native TJSEventHandler
addClass / removeClass	CSS class management
containsClass(cls)	Checks whether a CSS class is present
setFocus()	Moves browser focus to the element
Clear()	Clears innerHTML
getFile()	Returns the first file from an input[type=file]
Attrib[name]	Custom JSValue dictionary attached to the element
AsButton / AsInput / AsSelect / AsDiv	Typed casts to the underlying DOM element

TBaseForm

Subclass of **TWebComponent** that adds the contract required by PageManager for dynamic page loading.

```
  class function getTemplatePath(): String; virtual;
  // Template HTML path. Default: '' (PageManager then uses <base
folder>pages/<route>.html)

  class function loadNeededCSS(const aBasePath: String): Boolean; async; virtual;
  // Override to load page-specific CSS before rendering. aBasePath: application root
folder ('' or 'wms/').
```

```
  property BasePath: String;  // same folder, available to the instance; assigned
before BindUI
```

3.2 LayoutComponents — Page Structure

Manages the four fixed layout regions: header, footer, and the two side columns. Side columns include automatic responsive behaviour.

TLayoutComponent

Base class for header, footer, and columns. The **LoadContent(aPath)** method asynchronously fetches a remote HTML file and injects it into the root element.

TSideColumnComponent

Extends **TLayoutComponent** with:

- **Show / Hide / Toggle**: column visibility control
- Automatic resize listener: hides the column below 768 px viewport width
- Correct resize listener cleanup in the destructor (prevents dangling references)
- Fires the **`'layout:column-toggled'`** event on the EventsManager whenever visibility changes

3.3 PageManager — SPA Routing

The routing core. It maps route strings to **TBaseForm** classes, manages navigation, and keeps exactly one active form at a time.

Registering routes

```
PageManager.RegisterForm(
  'home',          // route name (= URL hash)
  THomeForm,       // class to instantiate
  0, 0             // MinWidth, MinHeight (0 = default for any screen size)
);

// Responsive variant: different form on wide screens
PageManager.RegisterForm('home', THomeFormDesktop, 1024, 0);

// Optional last parameters: template file and application root folder
PageManager.RegisterForm('home', THomeForm, 0, 0, 'pages/home.html', 'wms/');
```

Validation and redirect

```
PageManager.OnValidatePage :=
  function(aPage: String): String
  begin
    if not UserIsLoggedIn and (aPage <> 'login') then
      Result := 'login'   // redirect to login page
    else
      Result := aPage;
  end;
```

Programmatic navigation

```
gv_WebApplication.PageManager.NavigateTo('detail');
// Updates URL hash, pushes history entry, loads template, creates form
```

> *The HTML template is chosen in this order: getTemplatePath() of the form, if it returns a path (a complete path); otherwise the htmlFile given to RegisterForm; otherwise <base*

> *folder>pages/<route>.html, where the base folder is the basePath given to RegisterForm (empty for pages in the site root). The PageManager prevents race conditions with the FIsNavigating flag: while a page is loading, further NavigateTo calls are ignored, not queued.*

3.4 WebApplication — Application Shell

An abstract class representing the application as a whole. Subclass it and implement **RegisterRoutes**.

```
TMyApp = class(TWebApplication)
protected
  procedure RegisterRoutes(); override;
  procedure InitializeHeader; override; async;
end;

procedure TMyApp.InitializeHeader;
begin
  // Load header HTML from file
  AWait(FHeader.LoadContent('partials/header.html'));
  // bind any header events here
end;

procedure TMyApp.RegisterRoutes;
begin
  FPageManager.RegisterForm('home',   THomeForm);
  FPageManager.RegisterForm('list',   TListForm);
  FPageManager.OnValidatePage := @Self.ValidatePage;
end;

var app: TMyApp;

initialization
  app := TMyApp.Create(nil, nil, nil, nil);
  AWait(app.Run());

end.
```

Wake Lock API

Useful for kiosk or dashboard applications that must not go to sleep.

```
var app := TMyApp.Create(nil, nil, nil, nil);
AWait(app.RequestWakeLock());
app.SetupWakeLockAutoRenew(); // re-acquires on visibilitychange
AWait(app.Run());
```

3.5 EventsManager — Internal Pub/Sub

A global event system for decoupled communication between components. The global instance **`gv_EventsManager`** is created automatically at startup.

Usage

```
// Register a listener (typically inside BindUI)
gv_EventsManager.AddListener(
  'user:updated',
  Self,                          // owner (used by RemoveListenersFor)
  procedure(Sender: JSValue)
  begin
    Self.RefreshUserInfo(TJSObject(Sender));
  end
);

// Fire an event
```

```
gv_EventsManager.RaiseEvent('user:updated', aUserObject);

// Remove all listeners owned by a component (call in destructor)
gv_EventsManager.RemoveListenersFor(Self);
```

> *RaiseEvent operates on a snapshot copy of the listener list, so a handler may safely unsubscribe itself during execution. Exceptions thrown inside callbacks are not propagated to the raiser, so that one faulty handler cannot block the others (they are written to the browser console).*

3.6 InfiniteScroller — Progressive Loading

Displays very large lists without loading all elements at once. Uses a DocumentFragment to minimise DOM reflows during chunk rendering.

Setup and launch

```
FScroller := TInfiniteScroller.Create(
  ListContainerElement,    // scrollable TJSHTMLElement
  '',                      // inline template (empty = load from file)
  'templates/item.html'    // template file path
);
FScroller.ItemsPerChunk := 25;  // default: 20

FScroller.OnChunkRendered :=
  procedure(aStart, anEnd: Integer)
  begin
    // attach event listeners to newly rendered elements
  end;

FScroller.LoadData(FTotalCount, @Self.GetItemData);
```

GetItemData callback

```
function TListForm.GetItemData(const aIndex: Integer): TJSObject;
begin
  Result := TJSObject(FItems[aIndex]);
end;

// The HTML template uses {{key}} placeholders:
// <div class="card" data-id="{{id}}">
//   <h3>{{firstName}} {{lastName}}</h3>
//   <p>{{phone}}</p>
// </div>
```

3.7 Dialogs — Modal Dialogs

Non-blocking modal dialogs built dynamically in the DOM, with no external dependencies. Icons are embedded as base64 data URIs.

Signature: ShowMessage(aTitle, aMessage: String; aMsgType: TMessageType = mtInformation). Title and message are both required. The dialog has a single OK button; mtConfirm shows a success icon (it is not a yes/no question).

```
uses Dialogs;

// Informational dialog
ShowMessage('Information', 'Operation completed', mtInformation);

// Error dialog
ShowMessage('Error', 'Connection failed: timeout', mtError);

// Warning dialog
```

```
ShowMessage('Warning', 'Unsaved changes.', mtWarning);
```

4. CSS & Theming

The styling system is entirely based on CSS custom properties (variables), defined on **`:root`**. The active theme is selected via the **`site-style`** attribute on the HTML element.

site-style value	Description
light	Light theme: white background, black text, grey borders
dark	Dark theme: #202020 background, light text, dark borders
dany	Custom theme: semi-transparent background, red text, greyscale filter on body

Switching theme at runtime

```
// Switch to the dark theme
document.documentElement.setAttribute('site-style', 'dark');

// Read the current theme
var theme := document.documentElement.getAttribute('site-style');
```

Main CSS Variables

Variable	Usage
--color-primary / --color-primary-hover	Primary button colour and link colour
--header-bg / --footer-bg	Header and footer background
--form-bg / --form-border	Form card background and border
--modal-bg / --modal-text	Modal dialog background and text
--scrollbar-thumb / --scrollbar-thumb-hover	Custom scrollbar thumb
--font-main / --font-title	Body and title typefaces

Included CSS Files

- **basicAppStyle.css** — base variables, html/body reset
- **additionalStyles.css** — theme overrides (light, dark, dany)
- **appLayout.css** — 3-column flex layout
- **scrollbar.css** — custom WebKit scrollbar
- **forms.css** — centered form card styling
- **modalDialog.css** — animated overlay and dialog

5. Full Example: the "Address Book" App

The following complete example shows how to build a small SPA with two pages: a contact list with infinite scroll and a contact detail view.

5.1 Project Structure

```
addressbook/
├─ index.html
├─ css/
│   └─ (all framework CSS files)
├─ pages/
│   ├─ contacts.html     <- list template
│   └─ detail.html       <- detail template
├─ templates/
│   └─ contact-item.html <- single contact card template
├─ partials/
│   └─ header.html       <- header HTML
└─ pas/              <- Pascal sources
    ├─ AddressBookApp.pas
    ├─ ContactsForm.pas
    └─ DetailForm.pas
```

5.2 index.html

```
<!DOCTYPE html>
<html lang="en" site-style="dark">
<head>
  <meta charset="UTF-8">
  <title>Address Book</title>
  <link rel="stylesheet" href="css/basicAppStyle.css">
  <link rel="stylesheet" href="css/additionalStyles.css">
  <link rel="stylesheet" href="css/appLayout.css">
  <link rel="stylesheet" href="css/scrollbar.css">
  <link rel="stylesheet" href="css/forms.css">
  <link rel="stylesheet" href="css/modalDialog.css">
  <script src="addressbook.js"></script>  <!-- Pas2JS output -->
</head>
<body>
  <div id="app-container">
    <header id="app-header"></header>
    <main id="main-content">
      <aside id="left-column" class="hidden"></aside>
      <div id="center-container"></div>
      <aside id="right-column" class="hidden"></aside>
    </main>
    <footer id="app-footer" class="hidden"></footer>
  </div>
  <script>rtl.run();</script>
</body>
</html>
```

5.3 AddressBookApp.pas — Application Shell

```
unit AddressBookApp;

interface

uses WebApplication, PageManager, LayoutComponents,
     ContactsForm, DetailForm;

type
  TAddressBookApp = class(TWebApplication)
  protected
```

```
    procedure RegisterRoutes(); override;
    procedure InitializeHeader; override; async;
  end;

implementation

procedure TAddressBookApp.InitializeHeader;
begin
  AWait(FHeader.LoadContent('partials/header.html'));
end;

procedure TAddressBookApp.RegisterRoutes;
begin
  FPageManager.RegisterForm('contacts', TContactsForm);
  FPageManager.RegisterForm('detail',   TDetailForm);
end;

var app: TAddressBookApp;

initialization
  app := TAddressBookApp.Create(nil, nil, nil, nil);
  AWait(app.Run());

end.
```

5.4 pages/contacts.html — List Page Template

```
<div id="contacts-root" style="height:100%; display:flex; flex-direction:column">
  <h2 style="padding: 1rem; margin:0;">Contacts</h2>
  <div id="contact-list"
       style="flex:1; overflow-y:auto; padding: 0 1rem;">
    <!-- InfiniteScroller renders contact cards here -->
  </div>
</div>
```

5.5 templates/contact-item.html — Single Item Template

```
<div class="contact-card" data-id="{{id}}" style="
    border:1px solid var(--form-border);
    border-radius:6px; padding:0.8rem; margin-bottom:0.5rem;
    cursor:pointer; background:var(--form-bg)">
  <strong style="color:var(--color-text-dark)">{{firstName}} {{lastName}}</strong>
  <div style="color:var(--color-text); font-size:0.85rem">{{phone}}</div>
</div>
```

5.6 ContactsForm.pas — List with InfiniteScroller

```
unit ContactsForm;

interface

uses Web, JS, WebComponent, InfiniteScroller, EventsManager;

type
  TContactsForm = class(TBaseForm)
  private
    FScroller  : TInfiniteScroller;
    FContacts  : TJSArray;  // data loaded from API

    function GetContactData(const aIndex: Integer): TJSObject;
  protected
    procedure BindUI; override;
  public
    destructor Destroy; override;
  end;
```

```
implementation

procedure TContactsForm.BindUI;
var
  _ListEl : TJSHTMLElement;
begin
  // Populate data (in a real app: fetch from REST API)
  FContacts := TJSArray.new;
  FContacts.push(makeContact(1, 'John',  'Smith',  '+1 555 1234567'));
  FContacts.push(makeContact(2, 'Alice', 'Taylor', '+1 555 9876543'));
  // ... more contacts ...

  _ListEl := TJSHTMLElement(
    FRootElement.querySelector('#contact-list'));

  FScroller := TInfiniteScroller.Create(
    _ListEl, '', 'templates/contact-item.html');
  FScroller.ItemsPerChunk := 30;

  FScroller.OnChunkRendered :=
    procedure(aStart, anEnd: Integer)
    begin
      // attach click listeners to navigate to detail page
    end;

  FScroller.LoadData(FContacts.length, @Self.GetContactData);
end;

function TContactsForm.GetContactData(const aIndex: Integer): TJSObject;
begin
  Result := TJSObject(FContacts[aIndex]);
end;

destructor TContactsForm.Destroy;
begin
  FScroller.Free;
  gv_EventsManager.RemoveListenersFor(Self);
  inherited Destroy;
end;

end.
```

5.7 DetailForm.pas — Detail Page

```
unit DetailForm;

interface

uses Web, JS, WebComponent, EventsManager, PageManager;

type
  TDetailForm = class(TBaseForm)
  private
    procedure OnBackClick(aSender: TUIElementWrapper;
                          aEvent: TJSMouseEvent): Boolean;
  protected
    procedure BindUI; override;
  end;

implementation

uses WebApplication;

procedure TDetailForm.BindUI;
begin
  // Retrieve data passed via EventsManager
  var contactData: JSValue;
  contactData := TJSObject(gv_EventsManager)['_selectedContact'];
```

```
  Self.getElementById('fullname').textContent :=
    String(TJSObject(contactData)['firstName']) + ' ' +
    String(TJSObject(contactData)['lastName']);

  Self.getElementById('phone').textContent :=
    String(TJSObject(contactData)['phone']);

  Self.getElementById('btn-back').onclick := @Self.OnBackClick;
end;

function TDetailForm.OnBackClick(
  aSender: TUIElementWrapper; aEvent: TJSMouseEvent): Boolean;
begin
  Result := True;
  gv_WebApplication.PageManager.NavigateTo('contacts');
end;

end.
```

6. Quick API Reference

TWebComponent

Method / Property	Signature	Notes
Create	constructor(aRootElement: TJSHTMLElement)	Does not call BindUI
BindUI	procedure virtual	Override to bind UI and events; called by PageManager / LoadContent
getElementById	function(id: String): TUIElementWrapper	Lazy-init with internal cache
RootElement	property TJSHTMLElement	Read-only

TPageManager

Method / Property	Signature	Notes
RegisterForm	proc(route, class, minW, minH, htmlFile, basePath)	htmlFile and basePath are optional. Multiple registrations per route = responsive
NavigateTo	procedure(aRoute: string)	Async load + pushState
StartRouting	procedure	Called by Run(); reads initial hash
CurrentForm	property TBaseForm	Currently displayed form
OnValidatePage	property (write)	Hook for redirects and auth guards

TWebApplication

Method / Property	Signature	Notes
Create	constructor(aHeaderClass,	nil = use base class

Method / Property	Signature	Notes
	aFooterClass, aLeftColumnClass, aRightColumnClass: TLayoutComponentClass)	
Run	procedure async	Layout → Routes → Start
RegisterRoutes	procedure abstract	Must be overridden
InitializeHeader/Footer/Left/Right	procedure virtual async	Override to load content
RequestWakeLock	function: boolean async	Acquires screen wake lock
ReleaseWakeLock	procedure async	Releases wake lock
SetupWakeLockAutoRenew	procedure	Re-acquires on visibilitychange
PageManager	property TPageManager	Read-only

TjsEventsManager (gv_EventsManager)

Method	Signature	Notes
AddListener	proc(event, owner, callback)	Callback: procedure(Sender: JSValue)
RaiseEvent	proc(event, sender)	Invokes all registered listeners
RemoveListenersFor	proc(owner)	Call in component destructor
AddEvent	proc(event)	Pre-declare an event (optional)
HasEvent	function(event): Boolean	Checks whether the event exists

TInfiniteScroller

Member	Type / Signature	Notes
Create	constructor(container, template, templatePath)	Inline template takes precedence; if empty, the file is loaded. The row root must be accepted inside a (not tr/td/th/li/option)
LoadData	proc(totalCount, onGetData)	Renders first chunk immediately
Clear	procedure	Full reset; scroll listener is kept
ItemsPerChunk	property Integer (write)	Default: 20
ItemTemplate	property String (write)	Inline HTML template string
OnChunkRendered	property (write)	proc(startIdx, endIdx: Integer)
setTotalItemsCount	proc(aValue: Integer)	Update total count without reset

7. Other modules

uLBWebUtils — HTTP POST requests (TJSAsyncPOSTRequest always resolves: check ok(); HttpStatus and ErrorAnswer describe an error answer), loadResource / isCSSLoaded, HtmlEncode / TextToHTML / MarkdownLiteToHtml, getWSProtocol, convertToIsoDate.

TemplateEngine and uLBTemplateLoader — {{field}} templates with each / if / unless and partials. A template is read as text (LoadTemplate, LoadTemplateSections) and rendered before touching the DOM.

uLBTreeView — tree over nested data, with checkboxes and a context menu (a plain click raises no selection event).

uJSTabulator — Pascal wrapper of the Tabulator JavaScript library, which the project must load.

WSManager — one WebSocket connection with automatic reconnection; every message is republished on gv_EventsManager.

IndexedDBHelper, LeastRecentlyUsedCache — local persistence and an in-memory cache.

Full signatures and behaviour of every unit: FRAMEWORK_REFERENCE.md, in this folder.